

How Beautiful APIs Come to Life

Ladislav Dobrovský
ladislav.dobrovsky@gmail.com

(works at CEITEC BUT with HPC stuff)



python™



NumPy

django



pandas



Flask

web development,
one drop at a time



Numba





What is API?

- Application Programming Interface
- Wikipedia:
 - API is connection between computers or between computer programs. It is a type of software interface, offering a service to other pieces of software.

[Reddy, Martin (2011). API Design for C++]
 - **The interface to a software library is one type of API.**



What could make an API beautiful?

- Ease of use
 - Sensible defaults
 - Problem domain abstractions
 - Compatibility
 - Helpful error messages
- Choices encouraging good practices
- Documentation and examples
- Performance
- Personal taste and experience

● **Pythonic[ity]**
(only in Python™)



User's perspective: Method chaining

(properties and attributes can chain also)

```
df.assign(z=df.x * df.y).query("group == 'a']").z.sum()
```

Pandas

```
data -= data.mean(axis=1).reshape(n_samples, -1)
est = decomposition.PCA(5).fit(data)
```

*NumPy,
scikit-learn*

```
page = (html()
        .append(head()
                  .append(title().append_text('My Page'))
                  .append(meta(charset='utf-8'))))
        .append(body()
                  .append(h1().append_text('Hello World'))
                  .append(div(class_='container')
                          .append(p().append_text('Content here'))
                          .append(a(href='/link').append_text('Click me')))))
```

(Fluent API == ONLY chaining!)

PyHTML



User's perspective: Context managers

Basic: setup/cleanup

```
conn = pymysql.connect( ... )  
with conn:  
    with conn.cursor() as cursor:  
        cursor.execute(sql, (addr, passhash))
```

PyMySQL (implementing PEP 249)

Advanced: Distributed GPU computing

```
strategy =  
tf.distribute.MirroredStrategy()  
with strategy.scope():  
    model = get_compiled_model()  
    model.fit( ... )
```

Keras+TensorFlow

Advanced: chaining/nesting

```
doc = document(title='My page')  
with doc.head:  
    link(rel='stylesheet',  
href='style.css')  
with doc:  
    with div(id='header').add(ul()):  
        li(a('Home', href='/'))  
        li(a('About', href='/about'))  
    with div(id='content'):  
        h1('Welcome!')  
        p('!!!PAGE CONTENT!!!')
```

Dominate

User's perspective: Decorators

Registration, requirements

```
@app.route('/secret_page')
@login_required
def secret_page():
    return '秘密'
```

Flask

```
@app.task(bind=True, max_retries=3)
def process_data(self, data):
    try:
        # process
        do_the_job(data)
    except Exception as exc:
        raise self.retry(exc=exc, countdown=60)
```

Celery

Code compilation/generation

```
@jit(nopython=True,
     parallel=True)
def f(x, y):
    return x + y
```

Numba

```
@wp.kernel
def avg_kernel(
    a: wp.array(dtype=float),
    b: wp.array(dtype=float),
    out_avg: wp.array(dtype=float),
):
    i = wp.tid()
    out_avg[i] = (a[i] + b[i]) * 0.5
```

NVIDIA Warp

```
@cython.locals(a=cython.long,
               b=cython.long,
               n=cython.longlong)
def foo(a, b):
    n = a * b
    return n
```

Cython

User's perspective: Project's tree + magic imports

```
my_project/
├── manage.py
├── my_project/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── my_app/
    ├── migrations/
    │   └── __init__.py
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── urls.py
    └── views.py
```

```
# my_app/models.py
from django.db import models

class Question(models.Model):
    question_text = models.CharField(max_length=200)
    pub_date = models.DateTimeField("date published")
```

Django

... but really imported
nowhere

Django

in the project... **magic!**

Chaining + proxy objects



```
class KMeans(_BaseKMeans):
    # (...)

    def fit(self, X):
        # (...)
        return self

    def predict(self, X):
        # (...)
        return labels
```

```
y = KMeans(x_train).fit(x_new).predict()
```

scikit-learn

```
class DataFrame(NDFrame, OpsMixin):
    # (...)
    def groupby(self, by=None, (...)):
        return DataFrameGroupBy(
            obj=self, keys=by, (...))
    # (...)

class DataFrameGroupBy(GroupBy[DataFrame]):
    def aggregate(self, func=None, (...))
        # (...)
        op = GroupByApply(self, func, (...))
        result = op.agg()
        # (...)
        return result
```

agg = aggregate *# method alias*

```
df.groupby("A").agg(["min", "max"])
```

Pandas

Chaining: NumPy is C extension!



`my_array.T.shape`

```
PyTypeObject PyArray_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "numpy.ndarray",
    // (...)
    .tp_methods = array_methods,
    .tp_getset = array_getsetlist,
    // (...)
};

PyGetSetDef array_getsetlist[] = {
    {"T", (getter)array_transpose_get, /*...*/},
    {"shape", (getter)array_shape_get,
     (setter)array_shape_set, NULL, NULL},
};

static PyObject * array_transpose_get(
    PyArrayObject *self, void *) {
    return PyArray_Transpose(self, NULL);
}
```

```
PyObject *PyArray_Transpose(
    PyArrayObject *ap, PyArray_Dims *permute)
{
    // (...)
    Py_INCREF(PyArray_DESCR(ap));
    ret = (PyArrayObject *)
        PyArray_NewFromDescrAndBase(
            Py_TYPE(ap), PyArray_DESCR(ap),
            n, PyArray_DIMS(ap), NULL,
            PyArray_DATA(ap),
            flags, (PyObject *)ap, (PyObject *)ap);
    // (...)
    // fix the dimensions and strides of the return-array
    for (i = 0; i < n; i++) {
        /* (...) */
    }
    // (...)
    return (PyObject *)ret;
}
```

NumPy

Decorators - HOF syntax sugar

```
def decorator(f):  
    def wrapper(*args, **kwargs):  
        print('action before call')  
        r = f(*args, **kwargs)  
        print('action after call')  
        return r
```

return wrapper

```
def square(x):  
    return x*x
```

square = decorator(square)

→

```
@decorator  
def square(x):  
    return x * x
```

```
def ttt(z):  
    return z+z+z
```

ttt = decorator(decorator(ttt))

→

→

```
@decorator  
@decorator  
def ttt(z):  
    return z+z+z
```

Flask

Decorators with parameters

```
def gain(k):
    def gain_k(f):
        def wrapper(*args, **kwargs):
            r = k * f(*args, **kwargs)
            return r
        return wrapper
    return gain_k
```

Captured "k" → Closure

```
g1 = gain(5)      # wrapper with captured k=5
g2 = gain(3.14)   # wrapper with captured k=3.14
```

```
def square(x):
    return x*x
```

```
square = g1(g2(square))
```

```
@g1
@g2
def square(x):
    return x * x
```

```
@gain(5)
@gain(3.14)
def square(x):
    return x * x
```



**STOP USING
DECORATORS SO MUCH**

STOP DATING MY MOTHER!

**I'M GONNA DECORATE
EVEN HARDER!**

imgflip.com

Inside libraries even decorators are decorated!

```
from functools import update_wrapper
import typing as t
F = t.TypeVar("F", bound=t.Callable[ ..., t.Any])
```

```
@app.route('/xyz')
def the_page():
    return '<html>'
```

```
class App:    # (actually sansio.Scaffold → sansio.App → flask.App)
    @setupmethod
    def route(self, rule: str, **options) → t.Callable[[T_route],
        def decorator(f: T_route) → T_route:
            endpoint = options.pop("endpoint", None)
            self.add_url_rule(rule, endpoint, f, **options)
            return f
        return decorator
```

```
def setupmethod(f: F) → F:
    f_name = f.__name__
    def wrapper_func(self, *args, **kwargs):
        self._check_setup_finished(f_name)
        return f(self, *args, **kwargs)
    return t.cast(F, update_wrapper(wrapper_func, f))
```

*just register and return
the original function*

Flask

“Unrolled” without decorators

```
@app.route('/xyz')
def the_page():
    return '<html>'
```

```
@app.route('/signin', methods=['GET', 'POST'])
def login_page():
    return '<form>'
```

```
def the_page():
    return '<html>'
```

```
options = {} # possible route options
app._check_setup_finished('route')
endpoint = options.pop("endpoint", None)
app.add_url_rule(
    '/xyz', endpoint,
    the_page, **options)
```

```
def login_page():
    return '<form>'
```

```
options = {'methods': ['GET', 'POST']}
app._check_setup_finished('route')
endpoint = options.pop("endpoint", None)
app.add_url_rule(
    '/signin', endpoint,
    login_page, **options)
```

Introspection: Bytecode or AST

EXTREMELY simplified, CPU target

```
import ctypes
```

```
def jit(func):
```

```
    bytecode = func.__code__.co_code
```

```
    varnames = func.__code__.co_varnames # ( ... )
```

```
    llvm_ir = compile_to_LLVM_IR(  
        bytecode, varnames, ... )
```

```
    compiled_binary = llvm_compile(llvm_ir)
```

```
    def run_binary(*args):
```

```
        # ... convert args to C types ...
```

```
        functype = ctypes.CFUNCTYPE(  
            ctypes_restype, *ctypes_args)
```

```
        return functype(compiled_binary.address)
```

```
    return run_binary
```

```
@jit
```

```
def func(x, y):
```

```
    return x * y
```

Numba

EXTREMELY simplified

```
import ast, inspect
```

```
def kernel(func):
```

```
    filename = inspect.getsourcefile(func)
```

```
    _, fun_lineno = inspect.getsourcelines(func)
```

```
    source = inspect.getsource(func)
```

```
    tree = ast.parse(source)
```

```
    gpu_code = make_gpu_code(tree)
```

```
    def run_gpu_code():
```

```
        return do_the_gpu_stuff(gpu_code)
```

```
    return run_gpu_code
```

in another file:

```
import warp as wp
```

```
@wp.kernel
```

```
def func(x: wp.array(dtype=float),
```

```
        y: wp.array(dtype=float)):
```

```
    return x*y
```

NVIDIA Warp

Context manager: clever global state

```
class dom_tag: # all tags inherit dom_tag
```

```
    frame = namedtuple(
        'frame', ['tag', 'items', 'used'])
```

```
    _with_ctxs = defaultdict(list)
```

```
    def __init__(self):
```

```
        self._ctx = None
```

```
        self._add_to_ctx()
```

```
    def _add_to_ctx(self):
```

```
        thread_id = _get_thread_context()
```

```
        stack = dom_tag._with_ctxs.get(thread_id)
```

```
        if stack:
```

```
            self._ctx = stack[-1]
```

```
            stack[-1].items.append(self)
```

```
    def __enter__(self):
```

```
        thread_id = _get_thread_context()
```

```
        stack = dom_tag._with_ctxs[thread_id]
```

```
        stack.append((self, [], set()))
```

```
        return self
```

```
    def __exit__(self, type, value, traceback):
```

```
        thread_id = _get_thread_context()
```

```
        stack = dom_tag._with_ctxs[thread_id]
```

```
        frame = stack.pop()
```

```
        for item in frame.items:
```

```
            if item in frame.used: continue
```

```
            self.add(item)
```

```
        if not stack:
```

```
            del dom_tag._with_ctxs[thread_id]
```

```
doc = document(title='My page')
```

```
with doc:
```

```
    with div(id='header').add(ul()):
```

```
        li(a('Home', href='/'))
```

```
        li(a('About', href='/about'))
```

```
    with div(id='content'):
```

```
        h1('Welcome!')
```

```
        p('!!!PAGE CONTENT!!!')
```

Dominate

Import magicks

```
# app1/settings.py
BASE_DIR = \
    Path(__file__).resolve().parent.parent
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth', ... ]
```

```
def main(): # generated manage.py
    os.environ.setdefault(
        'DJANGO_SETTINGS_MODULE',
        'app1.settings')
    execute_from_command_line(sys.argv)
    # 'runserver' cmd → django.setup()
```

```
def setup(): # django/__init__.py
    from django.apps import apps
    from django.conf import settings
    apps.populate(settings.INSTALLED_APPS)
```

```
# django/conf/__init__.py SIMPLIFIED
ENVIRONMENT_VARIABLE = "DJANGO_SETTINGS_MODULE"
```

```
class Settings:
    def __init__(self):
        # (...)
        settings_module = os.environ.get(
            ENVIRONMENT_VARIABLE)
        self.SETTINGS_MODULE = settings_module
        mod = importlib.import_module(
            self.SETTINGS_MODULE)
        for setting in dir(mod):
            if setting.isupper(): # ONLY_UPPER_ATTRS
                setting_value = getattr(mod, setting)
                setattr(self, setting, setting_value)
```

```
# actually LazySettings() which wraps Settings()
settings = Settings()
```

```
# apps.populate and further calls use
# import_module and util.find_spec from importlib
# to import app1 and all submodules
```

Splitting imports from implementation

```
# keras/src/models/models.py
@keras_export(["keras.Model", "keras.models.Model"])
class Model(Trainer, base_trainer.Trainer, Layer):

# uses namex library

class keras_export(namex.export):
    def __init__(self, path):
        super().__init__(package="keras", path=path)

    def __call__(self, symbol):
        register_internal_serializable(self.path, symbol)
        return super().__call__(symbol)

# in generated keras/__init__.py
# keras/models/__init__.py
from keras.src.models.model import Model as Model
```

Keras

Nicer importing and error messages



```
# manual imports in pandas/__init__.py
from pandas.core.api import (
    ... , DataFrame, ... )
```

```
# in pandas/core/frame.py
@set_module("pandas")
class DataFrame(NDFrame, OpsMixin):
    ...
```

```
# in pandas/util/_decorators.py
def set_module(module):
    def decorator(func: F) → F:
        if module is not None:
            func.__module__ = module
        return func
    return decorator
```

```
# pandas/__init__.py

_hard_dependencies = ("numpy", "dateutil")

for _dependency in _hard_dependencies:
    try:
        __import__(_dependency)
    except ImportError as _e:
        raise ImportError(
            f"Unable to import required"
            f"dependency {_dependency}."
            " Please see the traceback "
            "for details."
        ) from _e
del _hard_dependencies, _dependency
```

Thanks for the attention.

Questions?

Honorary mentions:

AIOHTTP, attrs, Beautiful Soup, Click, Dask,
FastAPI, HTTPX, Kivy, openpyxl, Peewee,
Polars, Pyglet, PyQt, pytest, PyTorch,
Requests, Seaborn, Shapely, SimPy,
SQLAlchemy, statsmodels, SymPy, Xarray

Your favourite APIs?